

TU Delft-IMAP-IGS

Preference-Based Optimisation — IMAP Inter-Generational Solver (IGS) Framework

About

The open-source **Preferendus** software package for preference-based optimisation — used for all IMAP demonstrators in the ODESYS book — is publicly available via the GitHub repository: <https://github.com/TU Delft-Odesys>. The results of the first demonstrator (the floating wind farm) are described in Chapter 8.5 of the ODESYS book and can be accessed through the same repository. An extended stochastic variant is also available at: <https://github.com/Boskalis-python/ODYCON>.

A contemporary and further extended implementation of the **Preferendus** framework, incorporating multiple **Inter-Generational Solver (IGS)** instantiations — IGS-GA-I, IGS-GA-II, IGS-BRKGA, and IGS-SI — is available at: <https://github.com/TU Delft-IMAP-IGS>.

In particular, the **IGS-BRKGA** instantiation is used for the second demonstrator (dynamic vessel allocation — AlloDyn). The corresponding results are subject to confidentiality constraints and are available upon reasonable request.

Repositories

IGS-GA-I

<https://github.com/TU Delft-IMAP-IGS/IGS-GA-I>

The original IGS-GA algorithm — the foundational Inter-Generational Solver using a Genetic Algorithm backbone. Kareem's user-friendly implementation, equivalent to Harold's original version linked to all ODESYS book examples. Integrates IMAP preference aggregation directly into standard GA survival selection.

IGS-GA-II

<https://github.com/TU Delft-IMAP-IGS/IGS-GA-II>

An improvement on IGS-GA-I built on the NSGA-II genetic loop (crossover, mutation, mating), but replacing non-dominated sorting and crowding distance with IMAP affine aggregation survival. Designed for benchmarking against reference test suites (e.g. DAS-CMOP). Supports multi-stakeholder preference models, constraint handling via the Constraint Dominance Principle (CDP), tournament and truncation selection, and an optional cross-generation archive to stabilise IMAP Z-score normalisation.

IGS-BRKGA

<https://github.com/TU Delft-IMAP-IGS/IGS-BRKGA>

A Biased Random-Key Genetic Algorithm (BRKGA) instantiation of the IMAP-IGS framework, developed for combinatorial and constraint-heavy scheduling problems. Used as the solver for the AlloDyn dynamic vessel allocation demonstrator (Boskalis). Results from this demonstrator are subject to confidentiality and available upon reasonable request.

IGS-SI

<https://github.com/TUdelft-IMAP-IGS/IGS-SI>

A Swarm Intelligence (SI) instantiation of the IMAP-IGS framework targeting graph-based and routing optimisation problems. Implements the Preferendus method via Ant Colony Optimisation (ACO) with the same preference-aggregation interface as the GA-based solvers. Applicable to directed graph problems such as route planning and Markov-chain modelling.

Framework Overview

All IGS solvers share the same **IMAP (Integrative Maximisation of Aggregated Preferences)** preference-aggregation kernel:

1. Raw objective values are converted into preference scores (0–100) via stakeholder-defined preference functions.
2. Scores are Z-score-normalised across the current candidate pool.
3. A weighted sum P^* is computed over all criteria and stakeholders.
4. P^* is min-max-scaled to $[0, 100]$ to produce the final ranking.

Because normalisation is pool-relative, IMAP scores are generation-specific — a deliberate design property that drives diversity-preserving selection. For full terminology and formal definitions, see the **IMAP-IGS Terminology** document appended to this PDF.

References

Wolfert, A.R.M. et al. (2023). *Open Design Systems (ODESYS)*. TU Delft.

Repository: <https://github.com/TUdelft-Odesys>

Teuber et al. (2025). *ODYCON — Stochastic design-decision extension*.

Repository: <https://github.com/Boskalis-python/ODYCON>

Preferendus software = IMAP-IGS

with multiple IGS instantiations:

- IGS-GA: Harold's original one, using Tetra and linked with all Odesys book Examples, available at other Github (book related)
- IGS-GA-I: Kareem's GA user-friendly implementation (used in Boskalis etc., equals Harold's)
- IGS-GA-II: Lennard's NSGA-based implementation for reference test suites
- IGS-BRKGA: Lennard's CP-combinatorial Allodyn implementation
- IGS-SI: Emre's Graph-based optimization applications (e.g. routing or Markov) implementation

IMAP-IGS Terminology

IGS-GA-II: A GA which inherits the genetic loop (crossover, mutation, mating) from NSGA-II but replaces the non-dominated sorting and crowding distance with IMAP affine aggregation survival.

IMAP (Integrative Maximisation of Aggregated Preferences):

The preference aggregation kernel. Given a pool of N candidates, IMAP:

1. Converts each raw objective value into a preference score via stakeholder-defined preference functions.
2. Z-score-normalises those scores across the pool.
3. Computes a weighted sum (P^*) of the normalised scores.
4. Min-max-scales P^* to $[0, 100]$.

Because normalisation is pool-relative, IMAP scores cannot be compared across different pools or generations without re-normalising.

Survival Selection

IMAPSurvival: The two-pass survival operator that replaces NSGA-II's RankAndCrowdingSurvival.

Two-pass aggregation:

- First pass: runs IMAP over the full pool (parents + offspring) to identify and discard low-quality candidates (score ≤ 40).
- Second pass: runs IMAP over the surviving candidates only, producing the final scores used for selection.

Discard phase / threshold (score ≤ 40): After the first pass, any individual scoring ≤ 40 is removed from the candidate pool before the second pass. Skipped if it would leave fewer than N survivors. Under Constraint dominance principle (CDP), feasible individuals are always protected from discard when there are fewer feasible solutions than N.

Candidates: The pool that remains after the discard phase. The second-pass IMAF scores are computed on this set.

Truncation selection: Deterministically keeps the top N candidates ranked by final IMAF score. Under CDP the sort is lexicographic: feasible individuals rank above infeasible ones, then by IMAF score within each group.

Tournament selection: Runs N binary tournaments on the candidate population. Each tournament samples two candidates at random. The higher-scoring (and feasibility-preferred under CDP) candidate wins. Preserves more genetic diversity than truncation at the cost of weaker selection pressure.

Preference Model

Stakeholder: A decision-maker whose preferences are encoded as a set of preference functions over the objectives, plus a per-objective weight vector and an overall importance weight.

Stakeholder weights: Relative importance of each stakeholder. Normalised internally so they sum to 1.

Preference function: Function over an objective, mapping that objective to a preference score, reflecting the preference of a stakeholder.

Objective weights: Per-stakeholder importance of each objective. Should sum to 1 per stakeholder.

A-fine-aggregator

A-fine-aggregator: Core aggregation function. Steps:

1. Transpose p so rows = individuals, columns = criteria.
2. Z-score normalise each column: $z = (x - \mu) / \sigma$. If $\sigma = 0$, use $\sigma = 1e-6$ to avoid division by zero.
3. Compute $P^* = \sum w_i \cdot z_i$ (weighted sum over all criteria).
4. Min-max scale to $[0, 100]$. If all P^* values are identical, return 50 for all.

Z-score normalisation: Standardises each criterion to zero mean and unit variance across the pool. This ensures criteria with very different numerical ranges contribute equally to P^* .

P* (representative preference score): The weighted sum of Z-score-normalised preference values for one individual. The raw output of the aggregation before min-max scaling.

Constraint Handling

Constraint Dominance Principle (CDP): A feasible individual always beats an infeasible one in any comparison (survival or tournament), regardless of IMAP score. Applies whenever there are constraints.

Constraint Violation (CV): Total constraint violation of an individual: $CV = \sum \max(0, g_j)$ over all constraints. A solution is feasible if and only if $CV = 0$.

CV preference function: An explicit preference applied to each individual's total CV. When set, all stakeholder weights are scaled by $(1 - cv_weight)$ and the CV criterion receives weight cv_weight . When the population is fully feasible, all CV scores ≈ 100 and $std \approx 0$, so the CV term contributes essentially nothing.

CV weight: Fraction of total aggregation weight assigned to the CV criterion. Must be in $(0, 1)$ when `cv_pref_fn` is set.

Elitism and Best Tracking

Current best: The highest-scoring feasible individual seen across all completed survival passes. Injected into the pool at every generation to update its preference score with the new population. Updated only when a strictly better, genuinely different solution is found in the same generation's second-pass scoring context.

Current_best_gen: The generation index at which `current_best` was last updated.

Hall of fame: Ordered list of all outgoing `current_best` values. Every time `current_best` is replaced, the old one is appended here. Provides a history of all best-found solutions across the run. Note: general concept in GA's.

Elitism guarantee: Before each generation's selection call, `current_best` is injected into the population if not already present. After selection, `current_best` is force-kept in the survivors if it would otherwise be dropped.

Archive

Best archive: Optional storing of previously best individuals. When enabled, archive members are appended to the candidate pool's objective-value dict before the second-pass IMAP scoring. This extends the normalisation reference frame across generations, stabilising Z-score computation.

Archive stabilisation: Archive members participate in Z-score normalisation only; they are never returned as survivors. Including them prevents the score distribution from collapsing when the live candidate pool becomes too homogeneous.

Archive max size: Maximum archive size. When exceeded, we score all archive members against each other and discards the lower-scoring 50%, retaining at least one individual.

Inherited NSGA-II Infrastructure

Population size: Number of individuals maintained in the population across generations.

Population: Container for a set of Individual objects. Carries evaluated objective values (F), constraint values (G), decision variables (X), and any custom attributes (here: `imap_score`, `is_feasible`).

Individual: Single solution with fields X (decision variables), F (objectives), G (constraints), and arbitrary named attributes.

Crossover / Mutation: Standard pymoo operators inherited from NSGA2/GeneticAlgorithm; IMAP-GA does not modify them.